AgentForge Local AI

A Privacy Focused Offline AI Agent Powered by Small Language Models

Run locally. Use tools intelligently. Keep your data under your control.

Saqib Ahmad Bhat
Student Developer · B.E. Computer Science and Engineering · Government College of Engineering, Tirunelveli
AI/ML EWB internship course, under the supervision of the EWB Courses programme

Overview | Architecture | Pipeline | Security | Results | Contact

Problem statement and motivation



Cost

Most agent demos depend on paid cloud APIs and an API key.



Privacy

Prompts, notes and documents are sent to a third-party server.



Connectivity

No network, no agent: cloud-only demos stop working offline.



Opacity

Frameworks hide the loop, so it is hard to see how agents work.



Opportunity: small open-weight language models (about 1–4 billion parameters) now run on a laptop and support structured tool calling, so the whole agent can run, be inspected and be tested on one machine.

Objectives and scope

O1 Local inference only

No paid API, no cloud fallback.

O2 Safe tool execution

Model proposes; validated, allow-listed code executes.

O3 Bounded execution

Iteration, repeat and size limits.

O4 Explicit failures

Service down, missing model, timeouts, storage errors.

O5 Usable interfaces

Terminal and local web UI with tool traces.

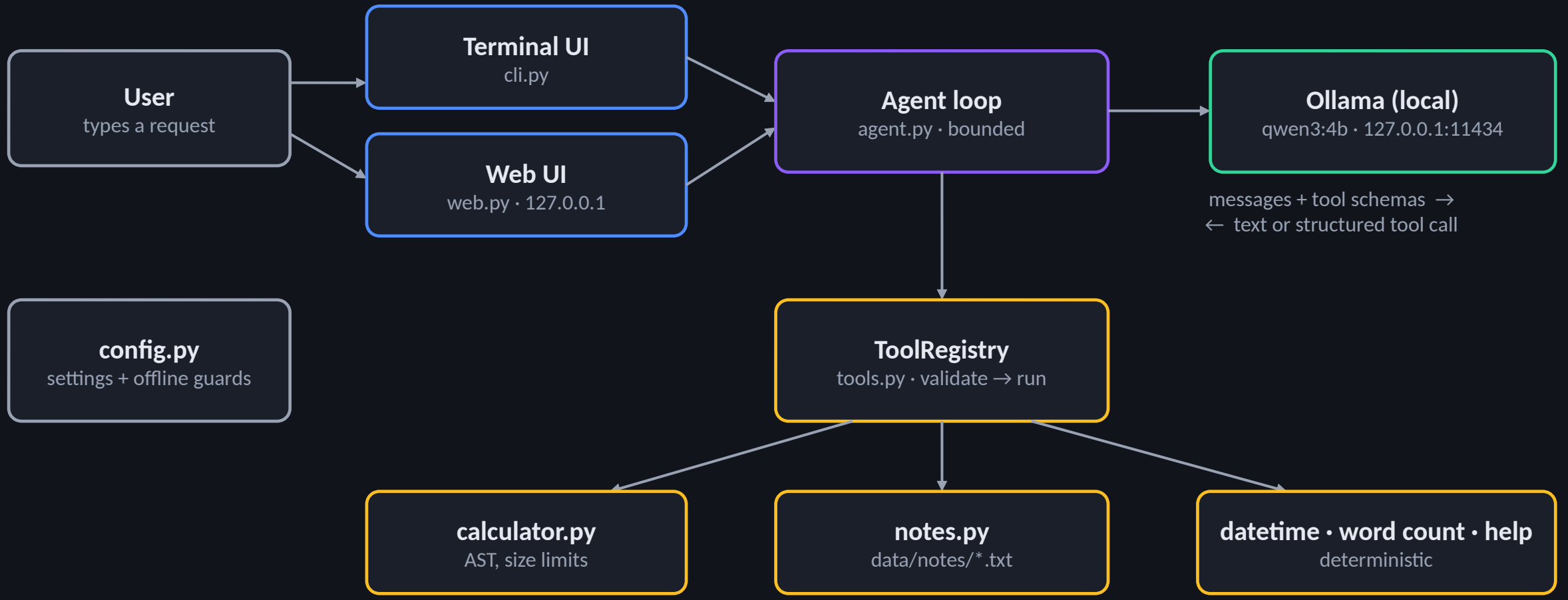
O6 Evidence

Tests without a model; docs match the code.

Out of scope

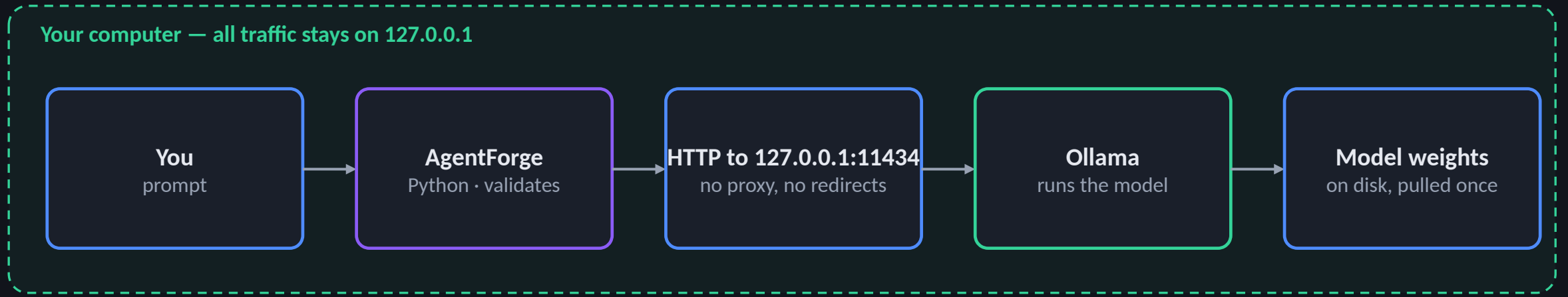
- Training or fine-tuning models
- Multi-user or cloud deployment
- Document retrieval beyond notes
- Model benchmarks (future work)

System overview and architecture



The model only proposes. The registry checks every call against the allow-list and validates its arguments before anything runs.

Offline inference pipeline and data flow



1

Receive

Reject empty or over-long input

2

Ask the model

Conversation + 8 tool schemas

3

Decide

No tool call → final answer

4

Validate

Allow-list, names, types, lengths

5

Execute

Deterministic Python tool

6

Observe & stop

Answer, 6 calls, or repeats

Model integration: what the code actually does

Ollama client (llm.py)

- host 127.0.0.1:11434 · timeout 180 s
- trust_env=False (proxies ignored), follow_redirects=False
- chat(model, messages, tools=8 schemas, think, num_predict=2048)
- Health check: /api/version (≥ 0.9.0) and /api/show capabilities (tools, thinking)
- Model list from /api/tags, cloud models excluded

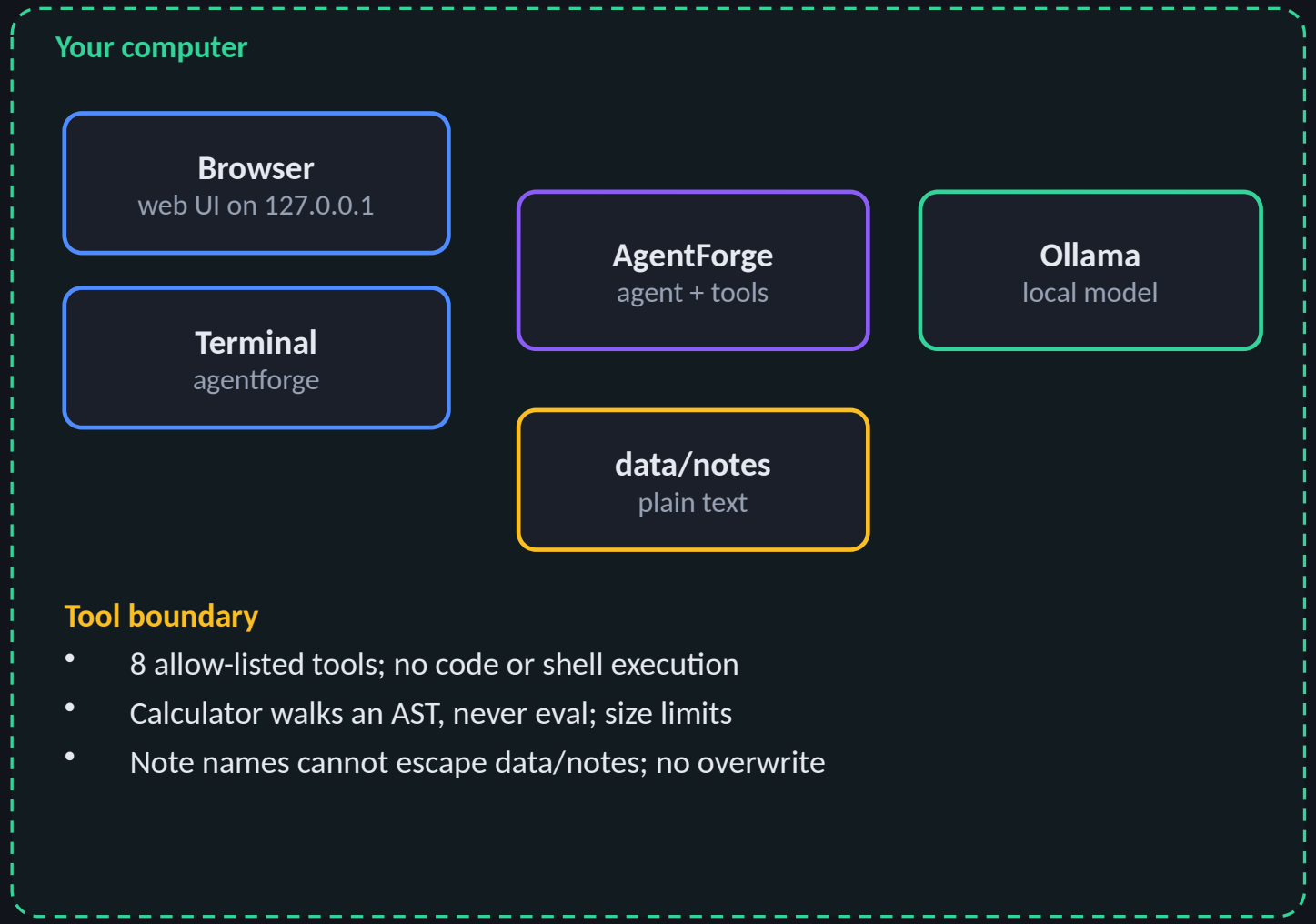
```
{"type": "function", "function": {
  "name": "calculator",
  "parameters": {"type": "object",
    "properties": {"expression": {"type": "string"}},
    "required": ["expression"]}}}
```

Low-level failure	AgentForge error	User hint
ConnectionError / transport	LLMUnavailableError	Start Ollama
httpx timeout	LLMTimeoutError	Retry / raise timeout
HTTP 404 model not found	ModelNotFoundError	ollama pull <model>
“does not support tools”	ModelCapabilityError	Use a tool-capable model
Bad JSON / schema mismatch	LLMResponseError	Malformed reply reported

No fallback: every failure becomes a specific error with a hint; nothing is ever sent to another service.

Tool calling is native (structured tool_calls from Ollama). Whether a model uses tools reliably must be checked with that model; --check reports its advertised capabilities.

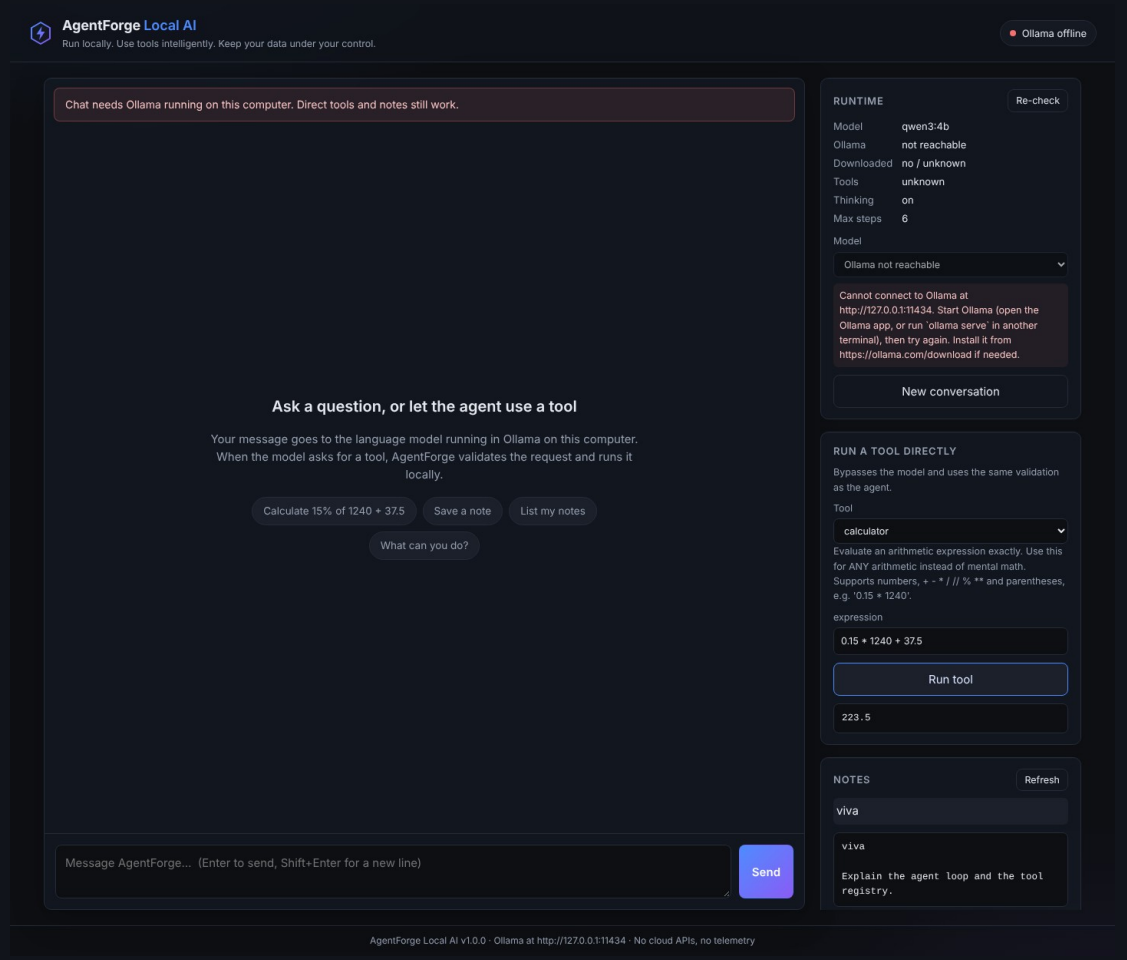
Privacy and security boundaries



Blocked at the boundary

- **Remote Ollama URL** refused unless explicitly allowed
- ***-cloud models** refused
- **Proxy variables** ignored for model traffic
- **Cloud fallback** none exists
- **Web UI requests** foreign Host / Origin → 403; JSON ≤ 64 KB; strict CSP

Genuine interface screenshots



Web UI, Ollama not running (real state). Chat is disabled with a clear message; the calculator and a note were used through the direct tool panel.

```
Terminal — AgentForge Local AI (captured output; Ollama not running; typed input shown after "You:")

$ agentforge --check
Model:              qwen3:4b
Ollama URL:         http://127.0.0.1:11434
Ollama reachable:   no
Model downloaded:   no / unknown
Thinking mode:      on
Max iterations:     6   Timeout: 180s
Notes folder:       <project>/data/notes
Problem: Cannot connect to Ollama at http://127.0.0.1:11434. Start Ollama (open the Ollama app, or run `ollama serve` in another terminal), then try again. Install it from https://ollama.com/download if needed.
(exit code 2)

$ agentforge
AgentForge Local AI v1.0.0  Run locally. Use tools intelligently. Keep your data under your control.
Model qwen3:4b via Ollama at http://127.0.0.1:11434 - no cloud APIs, notes stay on this computer.
Type a message, /help for commands, /quit to exit.
Problem: Cannot connect to Ollama at http://127.0.0.1:11434. Start Ollama (open the Ollama app, or run `ollama serve` in another terminal), then try again. Install it from https://ollama.com/download if needed.
Chat needs the model, but /run, /notes and /help still work.

You: /run calculator 0.15 * 1240 + 37.5
[calculator] 223.5
You: /run save_note {"name": "viva", "content": "explain the agent loop"}
[save_note] Saved note 'viva' (22 characters).
You: /run read_note viva
[read_note] explain the agent loop
You: /run shell {"command": "rm -rf ~"}
Rejected/failed: Unknown tool 'shell'. Allowed tools: calculator, save_note, read_note, list_notes, search_notes, current_datetime, word_count, get_help.
You: /run calculator 9**9**9**9
Rejected/failed: Exponents larger than 1000 are not allowed.
You: /quit
Bye.
```

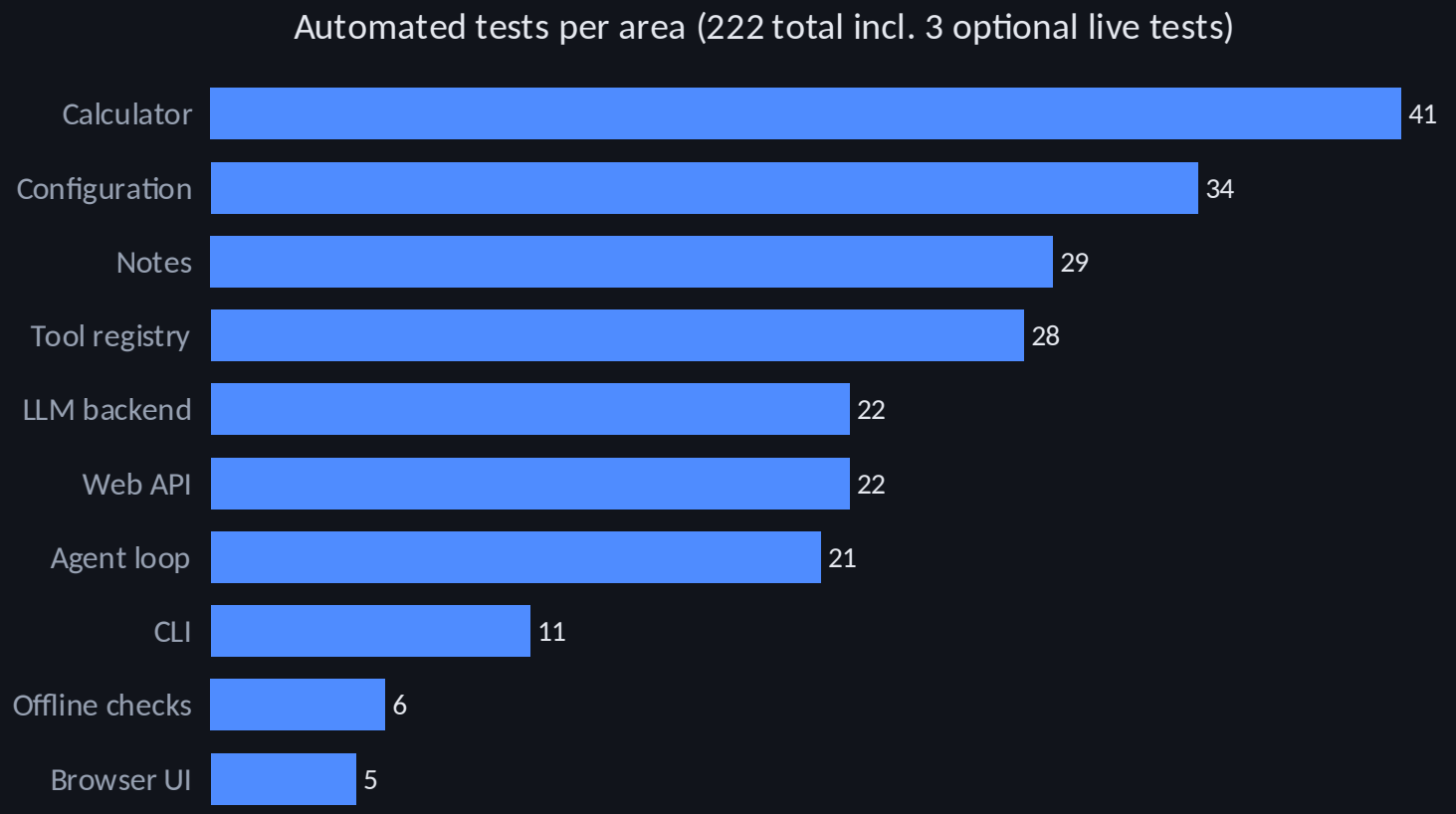
Terminal, captured output (Ollama not running). Unknown tool 'shell' rejected; 9**9**9**9 refused; notes saved and read.

Testing methodology and actual results

214 **core tests passed**
Python 3.10.12 and 3.12.15

219 **incl. 5 browser tests**
Python 3.13 + Chromium

0 **real-model runs**
not yet performed



Quality checks (29 Oct 2026): ruff lint and format clean · mypy clean · bandit: no issues · pip-audit: no known vulnerabilities. Real-model and live tests are still to be run.

Limitations and future improvements

Limitations

- Real-model behaviour not yet verified
- Small models may pick the wrong tool
- CPU inference can be slow
- Cloud-model detection is name-based
- Notes not encrypted; single user

Future improvements

- Measure tool-selection accuracy and latency (5 runs × 3 models)
- Note editing with confirmation
- Keyword search over a chosen folder
- Streaming responses
- Optional encryption of notes

Conclusion and references

A language model is good at deciding what to do. Exact, safe, auditable doing belongs in validated code. AgentForge keeps both on your own computer.

References

- S. Yao et al., “ReAct: Synergizing Reasoning and Acting in Language Models,” ICLR 2023, arXiv:2210.03629.
- T. Schick et al., “Toolformer: Language Models Can Teach Themselves to Use Tools,” NeurIPS 2023, arXiv:2302.04761.
- A. Yang et al. (Qwen Team), “Qwen3 Technical Report,” arXiv:2505.09388, 2025.
- Ollama, https://ollama.com (accessed October 2026).
- Upstream baseline: B. Kandala, Fully-Offline-AI-agent-using-SLM, MIT licence, commit ef2d5f5.
- Full report: docs/PROJECT_REPORT.md in the repository.

Maintainer and contact

Saqib Ahmad Bhat

Student Developer · B.E. Computer Science and Engineering · Government College of Engineering, Tirunelveli, Tamil Nadu, India

saqibahmadbhat885@gmail.com

linkedin.com/in/saqibahmadbhat

ORCID 0009-0003-9248-6174

github.com/SaqibAhmadBhat/agentforge-local-ai

saqibahmadbhat.github.io/agentforge-local-ai

Academic context: AI/ML EWB internship course, under the supervision of the EWB Courses programme.
Acknowledgement: builds on the MIT-licensed Fully-Offline-AI-agent-using-SLM by Bhargava Kandala; licence notice retained.